

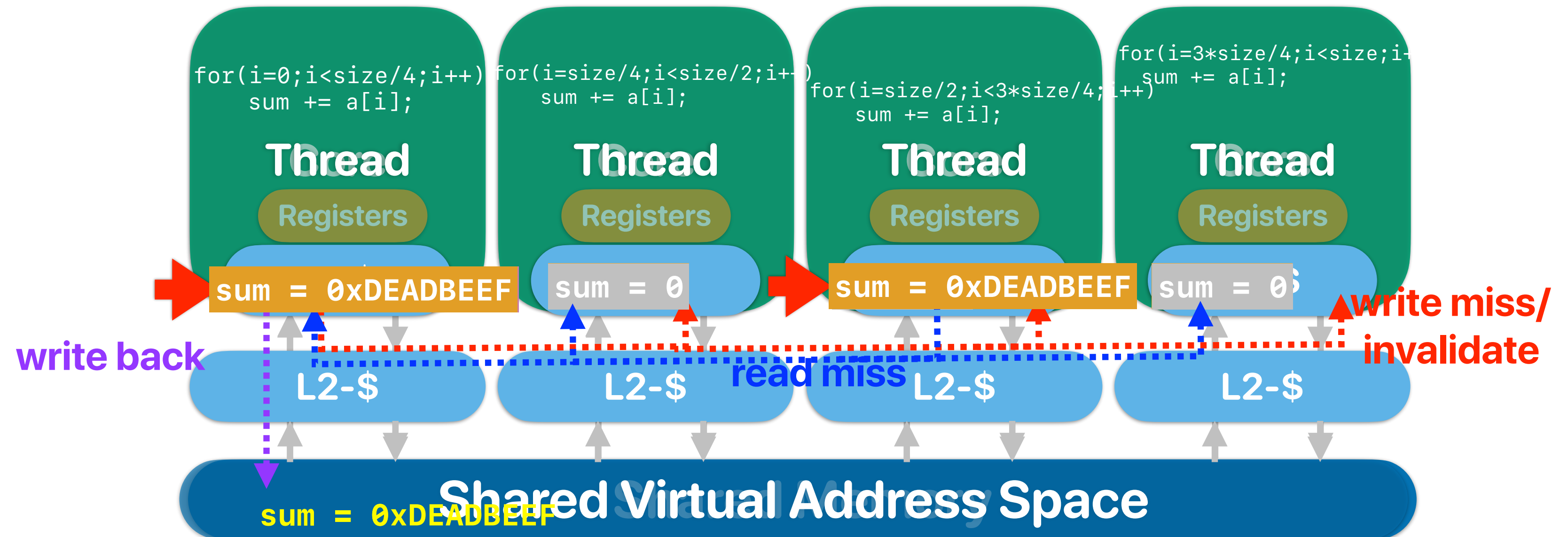
The Dark Silicon Era

Hung-Wei Tseng

Recap: Modern processors have both CMP/SMT



What happens when we write in coherent caches?



Recap: 4Cs of cache misses

- 3Cs:
 - Compulsory, Conflict, Capacity
- Coherency miss:
 - A "block" invalidated because of the sharing among processors.
- True sharing
 - Processor A modifies X, processor B also want to access X.
- False sharing
 - Processor A modifies X, processor B also want to access Y.
However, Y is invalidated because X and Y are in the same block!

Recap: Possible scenarios

```
int main() {
    int i;
    pthread_t thread[2];
    pthread_create(&thread[0], NULL, modifya, NULL);
    pthread_create(&thread[1], NULL, modifyb, NULL);
    pthread_join(thread[0], NULL);
    pthread_join(thread[1], NULL);
    fprintf(stderr, "(%d, %d)\n", x, y);
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

volatile int a,b;
volatile int x,y;
volatile int f;
void* modifya(void *z) {
    a=1;
    x=b;
    return NULL;
}
void* modifyb(void *z) {
    b=1;
    y=a;
    return NULL;
}
```

Thread 1

a=1;

x=b;

Thread 2

b=1;
y=a;

(1,1)

Thread 1

a=1;
x=b;

Thread 2

b=1;
y=a;

(1,0)

Thread 1

a=1;
x=b;

Thread 2

b=1;
y=a;

(0,1)

Thread 1

x=b;
a=1;

Thread 2

y=a;

b=1;

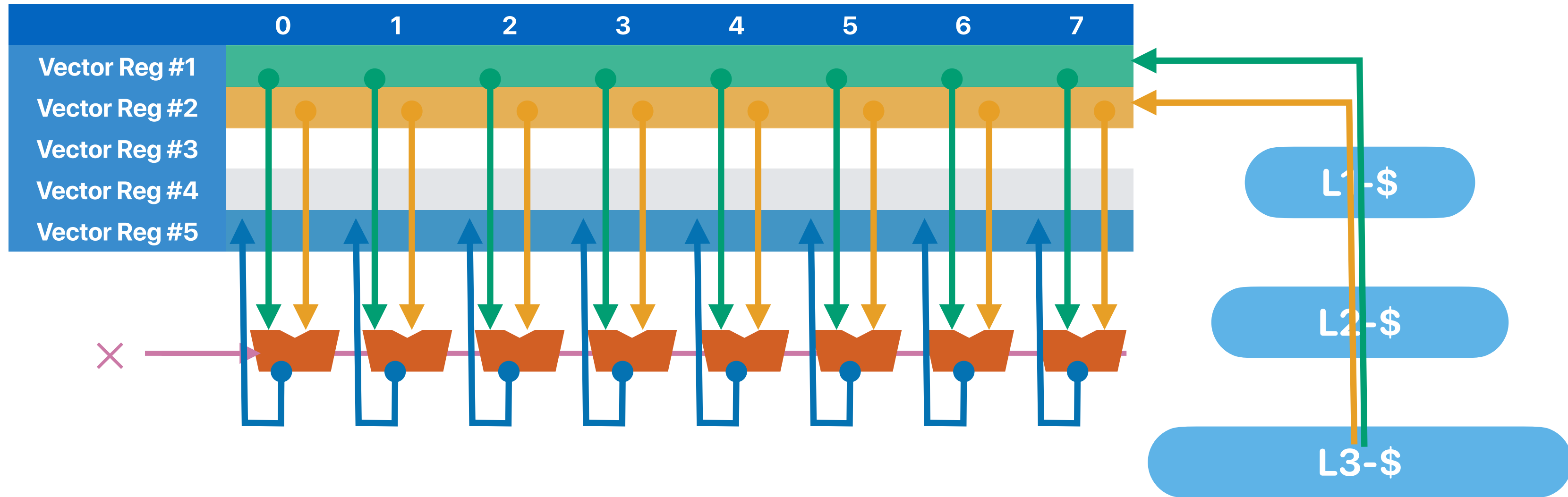
(0,0)

OoO Scheduling!

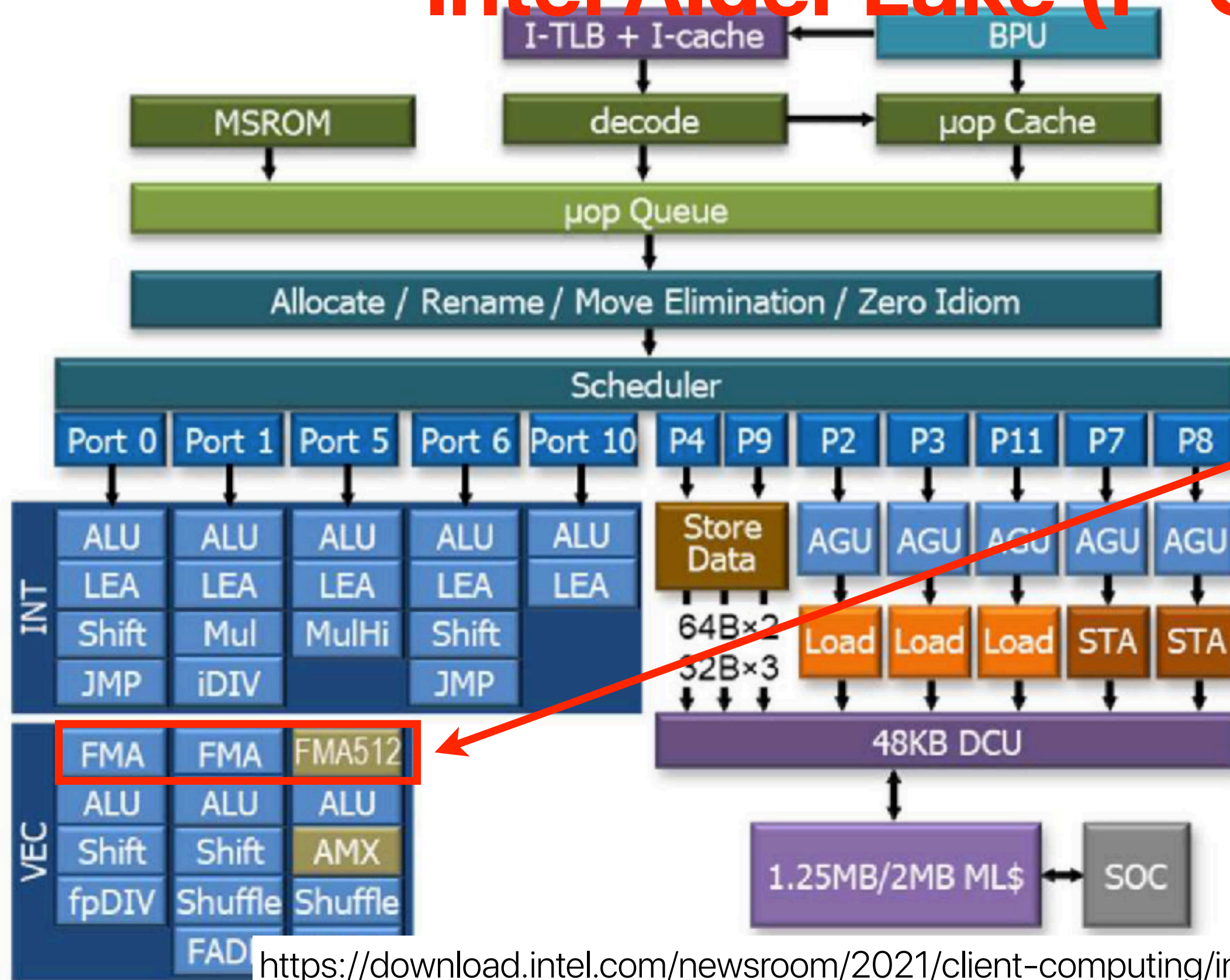
Recap: parallel programming

- Cache coherency only guarantees that everyone would eventually have a coherent view of data, but not when
- Cache coherency may create unexpected cache invalidations/misses if you do it wrong
- Processor behaviors are non-deterministic
 - You cannot predict which processor is going faster
 - You cannot predict when OS is going to schedule your thread
 - You cannot predict when the processor is going to schedule an instruction
- Cache consistency is hard to support

Vector processing architecture



Intel Alder Lake (P-Core)



**AVX Vector
Processing Units**

Summary: Parallelism in modern computers

- Instruction-level parallelism — perform various, independent instructions simultaneously
 - Pipeline
 - OoO/Superscalar
- Data-level parallelism — perform the same operation on multiple data elements in parallel
 - SIMD instructions
- Thread-level parallelism — perform independent computation streams (composed of many instructions or SIMD instructions)
 - Multicore/SMT processors

Recap — Take-aways: parallel programming

- Cache coherency only guarantees that everyone would eventually have a coherent view of data, but not when
- Cache coherency may create unexpected cache invalidations/misses if you do it wrong
- Processor behaviors are non-deterministic
 - You cannot predict which processor is going faster
 - You cannot predict when OS is going to schedule your thread
 - You cannot predict when the processor is going to schedule an instruction
- Cache consistency is hard to support

Outline

- Processors in data centers
- The dark silicon problem
- Challenges and state-of-the-art solutions in the dark silicon era

Multithreaded Architectures in Google Search

Web search for a planet: The Google cluster architecture

Luiz Andre Barroso, Jeffery Dean, Urs Holzle

Google

IEEE Micro 2003

Google Datacenter Goals

- *Price/performance beats peak performance.* We purchase the CPU generation that currently gives the best performance per unit price, not the CPUs that give the best absolute performance.
- *Using commodity PCs reduces the cost of computation.* As a result, we can afford to use more computational resources per query, employ more expensive techniques in our ranking algorithm, or search a larger index of documents.
- *Software reliability.* We eschew fault-tolerant hardware features such as redundant power supplies, a redundant array of inexpensive disks (RAID), and high-quality components, instead focusing on tolerating failures in software.
- *Use replication for better request throughput and availability.* Because machines are inherently unreliable, we replicate each of our internal services across many machines. Because we already replicate services across multiple machines to obtain sufficient capacity, this type of fault tolerance almost comes for free.



What kind of processors Google search needs

- If we are designing a processor just for Google search or similar type of applications, how many of the following targets/features would fulfill the demand?
 - ① Can execute many instructions from the same process/thread simultaneously
 - ② Can execute many processes/threads simultaneously
 - ③ Can predict branch outcome accurately
 - ④ Have very large cache capacity

A. 0
B. 1
C. 2
D. 3
E. 4



What kind of processors Google search needs

- If we are designing a processor just for Google search or similar type of applications, how many of the following targets/features would fulfill the demand?
 - ① Can execute many instructions from the same process/thread simultaneously
 - ✓ ② Can execute many processes/threads simultaneously
 - ✓ ③ Can predict branch outcome accurately
 - ④ Have very large cache capacity

A. 0

B. 1

C. 2

D. 3

E. 4

given how little ILP our application yields, and shorter pipelines would reduce or eliminate branch mispredict penalties. The avail-

For such workloads, a memory system with a relatively modest sized L2 cache, short L2 cache and memory latencies, and longer (perhaps 128 byte) cache lines is likely to be the most effective.

ions concurrently and has superior branch prediction logic. In essence, there isn't that much exploitable instruction-level parallelism (ILP) in the workload. Our measurements suggest that the level of aggressive out-of-order, speculative execution present in modern processors is already beyond the point of diminishing performance returns for such programs.

end ones. Exploiting such abundant thread-level parallelism at the microarchitecture level appears equally promising. Both simultaneous multithreading (SMT) and chip multiprocessor (CMP) architectures target thread-level parallelism and should improve the performance of many of our servers. Some early

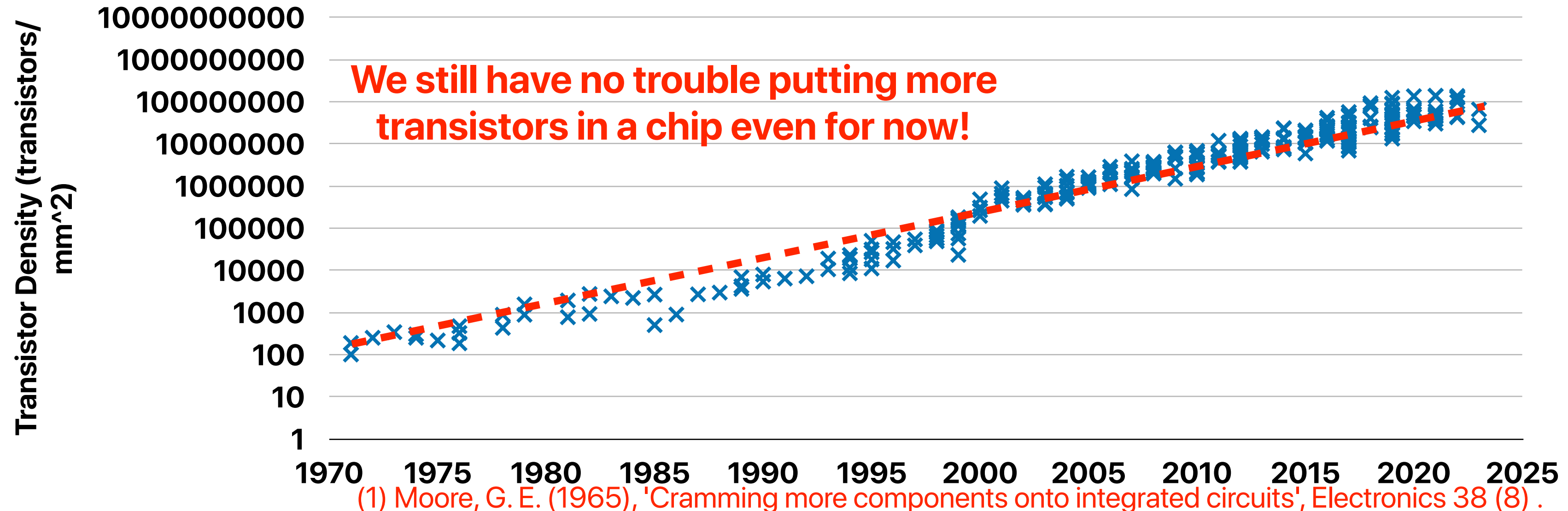
Metrics we care about data center design

- Costs — machine architecture, distributed system architecture, replication strategies
- Power — machine architecture
- Energy — machine architecture
- Space-efficiency — erasure coding, replication, distributed
- Throughput — replication, distributed
- Reliability — replication

What is and why is dark silicon?

Moore's Law⁽¹⁾

- The number of transistors we can build in a fixed area of silicon doubles every 12 ~ 24 months.
- Moore's Law "was" the most important driver for historic CPU performance gains





What happens if power doesn't scale with process technologies?

- If we are able to cram more transistors within the same chip area (Moore's law continues), but the power consumption per transistor remains the same. Right now, if put more transistors in the same area because the technology allows us to. How many of the following statements are true?
 - ① The power consumption per chip will increase
 - ② The power density of the chip will increase
 - ③ Given the same power budget, we may not able to power on all chip area if we maintain the same clock rate
 - ④ Given the same power budget, we may have to lower the clock rate of circuits to power on all chip area

A. 0
B. 1
C. 2
D. 3
E. 4

A screenshot of a poll interface. It shows five empty input boxes, each preceded by a letter (A, B, C, D, E) in a small font. The boxes are arranged vertically. The interface is simple and clean, with a light gray background.

What happens if power doesn't scale with process technologies?

- If we are able to cram more transistors within the same chip area (Moore's law continues), but the power consumption per transistor remains the same. Right now, if put more transistors in the same area because the technology allows us to. How many of the following statements are true?
 - ① The power consumption per chip will increase
 - ② The power density of the chip will increase
 - ③ Given the same power budget, we may not able to power on all chip area if we maintain the same clock rate
 - ④ Given the same power budget, we may have to lower the clock rate of circuits to power on all chip area

A. 0
B. 1
C. 2
D. 3
E. 4

Power consumption to light on all transistors

Chip

1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1

=49W

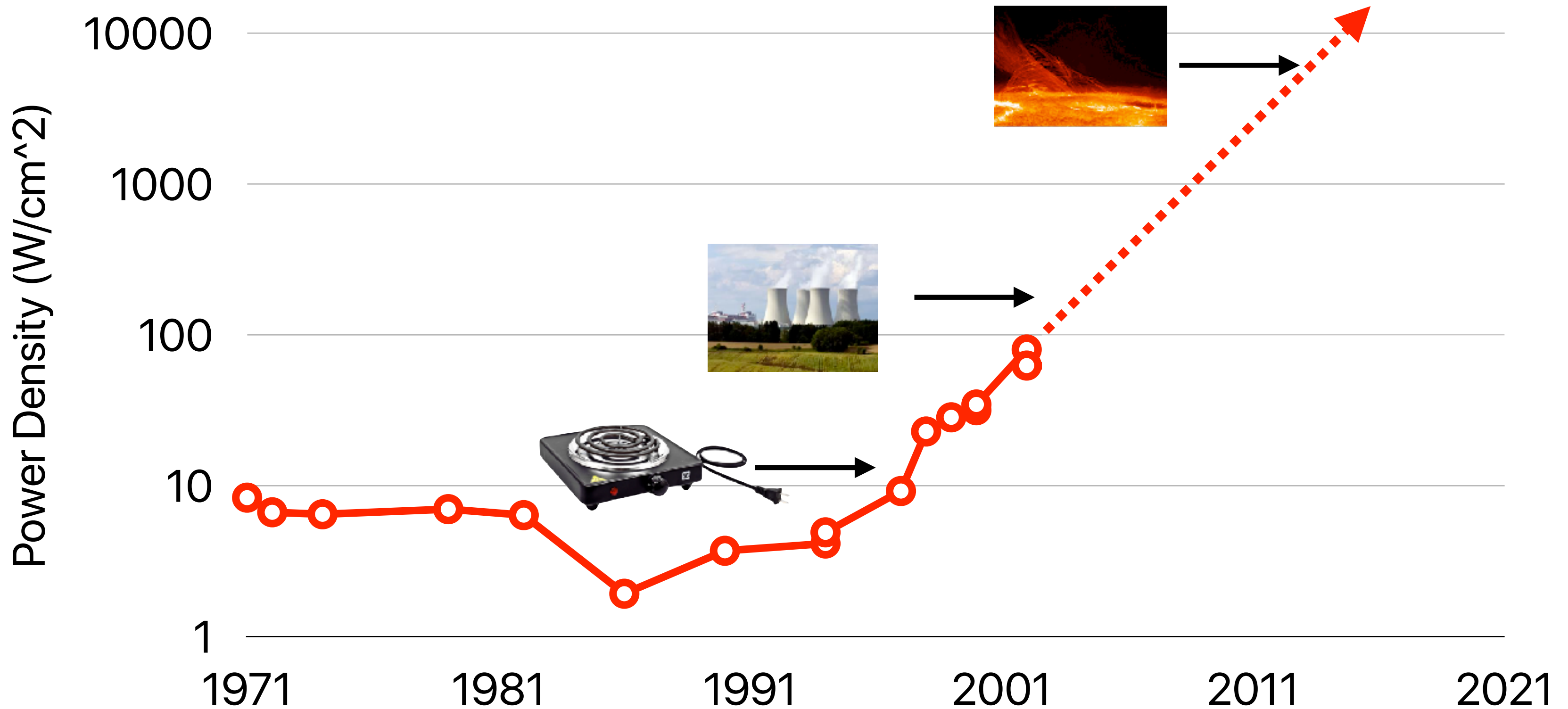
Dennardian Broken

Chip

1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1

=100W!

Power Density of Processors



Power consumption & power density

- $P_{dynamic} \sim \alpha \times C \times V^2 \times f \times N$
 - α : average switches per cycle
 - C : capacitance
 - V : voltage
 - f : frequency, usually linear with V
 - N : the number of transistors
 - $P_{leakage} \sim N \times V \times e^{-V_t}$
 - N : number of transistors
 - V : voltage
 - V_t : threshold voltage where transistor conducts (begins to switch)
 - Power density:
- 

Lower the frequency can reduce dynamic power!

$$P_{density} = \frac{P}{area}$$

If we have a fixed power budget

Slowdown all of them

Dark silicon!

Chip

1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1

=49W

Chip

0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5

Low frequency
~ 0.5W

=50W!

Chip

1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1

On ~
50W

Off ~
0W

Dark!

=50W!

What happens if power doesn't scale with process technologies?

- If we are able to cram more transistors within the same chip area (Moore's law continues), but the power consumption per transistor remains the same. Right now, if put more transistors in the same area because the technology allows us to. How many of the following statements are true?
 - ① The power consumption per chip will increase
 - ② The power density of the chip will increase
 - ③ Given the same power budget, we may not be able to power on all chip area if we maintain the same clock rate — **even if we put more cores, we cannot have all of them on!**
 - ④ Given the same power budget, we may have to lower the clock rate of circuits to power on all chip area — **or we have to operate all of them at a slower speed**

A. 0
B. 1
C. 2
D. 3
E. 4

$$\text{as } P_{dynamic} \sim \alpha \times C \times V^2 \times f \times N$$



Lower frequency

- Regarding lower the operating frequency when running applications, how many of the following statements are correct?
 - ① Lowering the frequency helps the same battery capacity to carry out more tasks
 - ② Lowering the frequency helps reduce the heat generation
 - ③ Lowering the frequency helps reduce the electricity bill/total energy consumption of running the same amount of tasks
 - ④ A CPU operating at 30% of the peak frequency can still consume more than 50% of the peak power

A. 0
B. 1
C. 2
D. 3
E. 4



Lower frequency

- Regarding lower the operating frequency when running applications, how many of the following statements are correct?
 - ① Lowering the frequency helps the same battery capacity to carry out more tasks
 - ② Lowering the frequency helps reducing the heat generation
 - ③ Lowering the frequency helps reducing the electricity bill/total energy consumption of running the same amount of tasks
 - ④ A CPU operating at 30% of the peak frequency can still consume more than 50% of the peak power
- A. 0
B. 1
C. 2
D. 3
E. 4

Power v.s. Energy

- Power is the direct contributor of "heat"
 - Packaging of the chip
 - Heat dissipation cost
 - Dynamic power
 - Leakage power
- $\text{Energy} = \text{Power} \times \text{Execution_Time}$
 - The electricity bill and battery life is related to energy!
 - Lower power does not necessary means better battery life if the processor slow down the application too much

Power consumption & power density

- $P_{dynamic} \sim \alpha \times C \times V^2 \times f \times N$

- α : average switches per cycle

- C : capacitance

- V : voltage

- f : frequency, usually linear with V

- N : the number of transistors

- $P_{leakage} \sim N \times V \times e^{-V/V_t}$

- N : number of transistors

- V : voltage

- V_t : threshold voltage where transistor conducts (begins to switch)

- Power density:

$$P_{density} = \frac{P}{area}$$

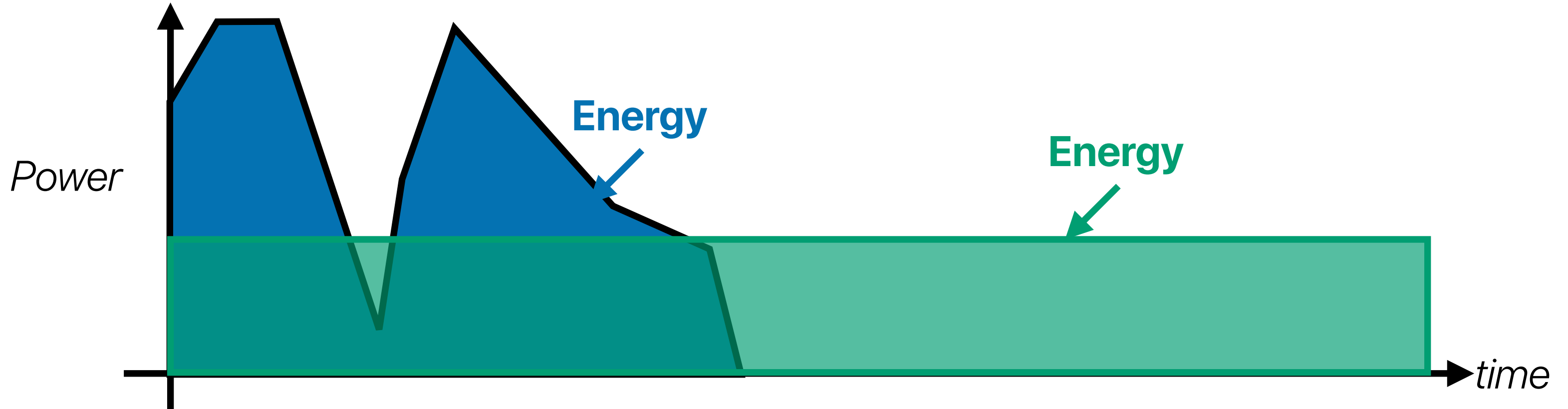
Dennard scaling discontinued — we cannot make voltage lower

Moore's Law allows higher frequencies as transistors are smaller

Moore's Law makes this smaller

Power v.s Energy

The Green can be more if power is not low enough



Recap: Demo — changing the max frequency and performance

- Change the maximum frequency of the intel processor — you learned how to do this when we discuss programmer's impact on performance
- LIKWID a profiling tool providing power/energy information
 - `likwid-perfctr -g ENERGY [command_line]`
 - Let's try blockmm and popcount and see what's happening!

Power v.s. Energy

- Power is the direct contributor of "heat"
 - Packaging of the chip
 - Heat dissipation cost
 - Dynamic power
 - Leakage power
- $\text{Energy} = \text{Power} \times \text{Execution_Time}$
 - The electricity bill and battery life is related to energy!
 - Lower power does not necessary means better battery life if the processor slow down the application too much

Power consumption & power density

- $P_{dynamic} \sim \alpha \times C \times V^2 \times f \times N$

- α : average switches per cycle

- C : capacitance

- V : voltage

- f : frequency, usually linear with V

- N : the number of transistors

- $P_{leakage} \sim N \times V \times e^{-V_t}$

- N : number of transistors

- V : voltage

- V_t : threshold voltage where transistor conducts (begins to switch)

- Power density:

$$P_{density} = \frac{P}{area}$$

**Lower the
frequency can
only reduce
active power**

**Moore's Law allows higher
frequencies as transistors are smaller**

Moore's Law makes this smaller

If we have a fixed power budget

Slowdown all of them

Dark silicon!

Chip

1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1

=49W

Chip

0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5

Low frequency
~ 0.5W

=50W!

Chip

1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1

On ~
50W

Off ~
0W

Dark!

=50W!

The frequency has to be really low!!!

Lower frequency

- Regarding lower the operating frequency when running applications, how many of the following statements are correct?
 - ① Lowering the frequency helps the same battery capacity to carry out more tasks
 - ✓ ② Lowering the frequency helps reducing the heat generation
 - ③ Lowering the frequency helps reducing the electricity bill/total energy consumption of running the same amount of tasks
 - ✓ ④ A CPU operating at 30% of the peak frequency can still consume more than 50% of the peak power
- A. 0
B. 1
C. 2
D. 3
E. 4

Take-aways: Challenges and SOTA solutions in the dark silicon era

- Even if we can address all programming challenges, multi-core performance has stopped to scale due to the Dark Silicon problem

Dark silicon problem

Dark silicon problem

- We are given the same area, twice amount of transistors
- We are given the same power budget
- We are given a certain applications
- How can we maximize the performance for these applications within the given constraints?

Challenges and state-of-the-art solutions in the dark silicon era

Given the same power budget, maximize the efficiency per chip

Some faster,
some slower

Some at top speed,

Slowdown all of them some are not functioning

Chip

0.7	0.7	0.7	0.7	0.7	0.3	0.3	0.3	0.3	0.3
0.7	0.7	0.7	0.7	0.7	0.3	0.3	0.3	0.3	0.3
0.7	0.7	0.7	0.7	0.7	0.3	0.3	0.3	0.3	0.3
0.7	0.7	0.7	0.7	0.7	0.3	0.3	0.3	0.3	0.3
0.7	0.7	0.7	0.7	0.7	0.3	0.3	0.3	0.3	0.3
0.7	0.7	0.7	0.7	0.7	0.3	0.3	0.3	0.3	0.3
0.7	0.7	0.7	0.7	0.7	0.3	0.3	0.3	0.3	0.3
0.7	0.7	0.7	0.7	0.7	0.3	0.3	0.3	0.3	0.3
0.7	0.7	0.7	0.7	0.7	0.3	0.3	0.3	0.3	0.3
0.7	0.7	0.7	0.7	0.7	0.3	0.3	0.3	0.3	0.3

Higher frequency ~ 0.7W
Very low frequency ~ 0.3W

=50W!

Chip

0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5

Low frequency ~ 0.5W

=50W!

Chip

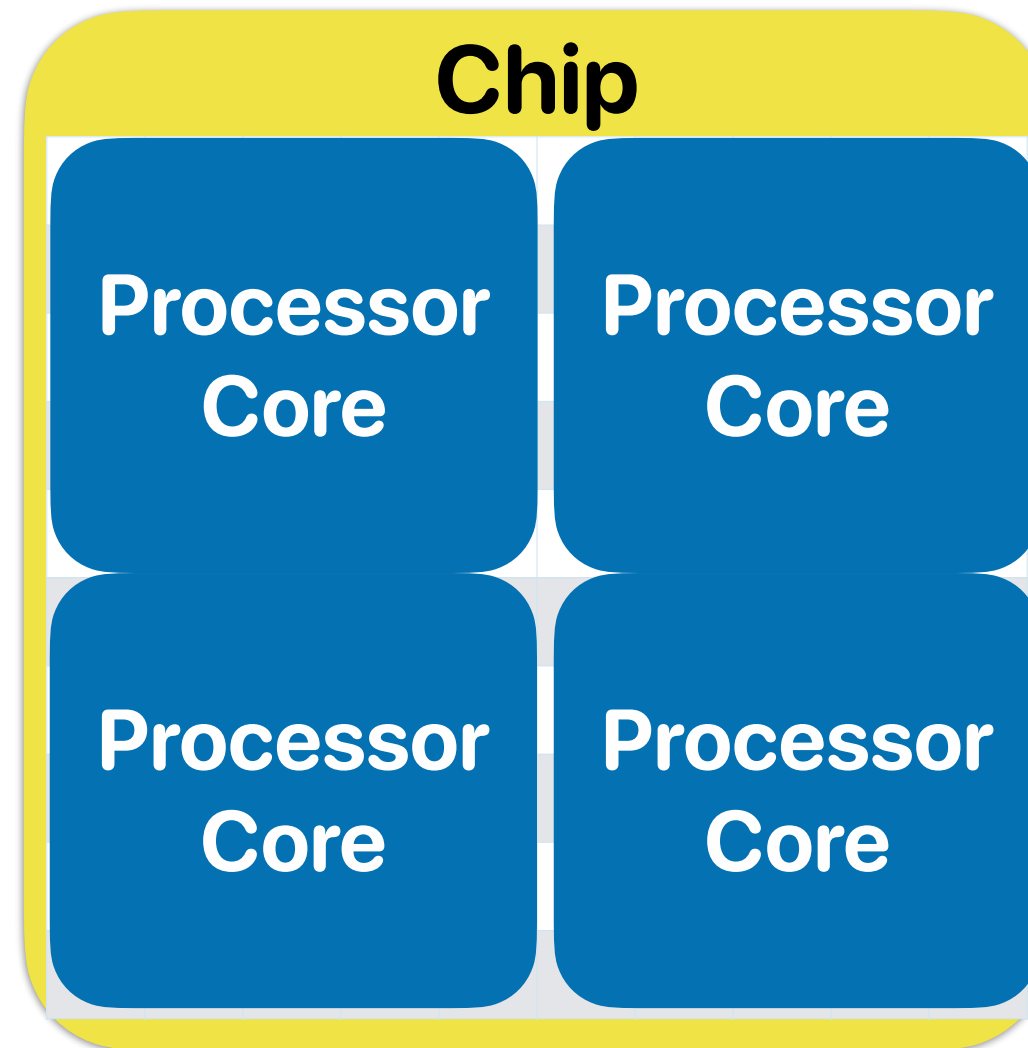
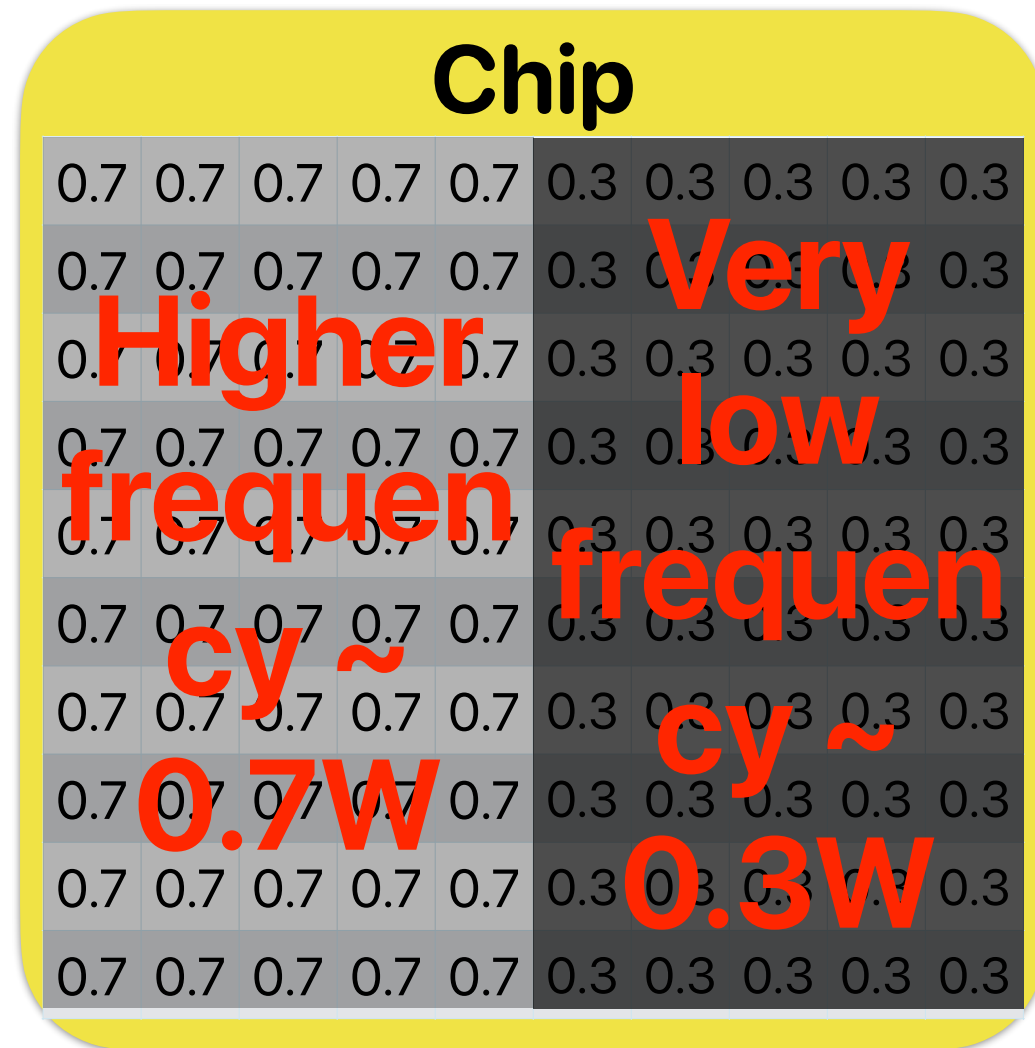
1	1	1	1	1	Off ~ 0W Dark!				
1	1	1	1	1					
1	1	1	1	1					
1	1	1	1	1					
1	1	1	1	1					
1	1	1	1	1					
1	1	1	1	1					
1	1	1	1	1					
1	1	1	1	1					
1	1	1	1	1					

On ~ 50W

=50W!

Given the same power budget, maximize the efficiency per chip

**Some faster,
some slower**



**Aggressively adjust
the frequency of
processor cores — If
only one program is
compute-intensive,
having it running at
full speed, others at
lower frequency or
turning off**

Modern processor's frequency

```
Socket(s): 1
NUMA node(s): 1
Vendor ID: GenuineIntel
CPU family: 6
Model: 151
Model name: 12th Gen Intel(R) Core(TM) i7-12700KF
Stepping: 2
CPU MHz: 1226.409
CPU max MHz: 5000.0000
CPU min MHz: 800.0000
BogoMIPS: 7219.20
```

i7-12700K

Intel 7

\$450.00 - \$460.00

PC/Client/Tablet, Workstation

CPU Specifications

Total Cores ⓘ	12
# of Performance-cores	8
# of Efficient-cores	4
Total Threads ⓘ	20

Max Turbo Frequency ⓘ	5.00 GHz
Intel® Turbo Boost Max Technology 3.0 Frequency ¹ ⓘ	5.00 GHz
Performance-core Max Turbo Frequency ⓘ	4.90 GHz
Efficient-core Max Turbo Frequency ⓘ	3.80 GHz
Performance-core Base Frequency	3.60 GHz
Efficient-core Base Frequency	2.70 GHz

Cache ⓘ 25 MB Intel® Smart Cache

Modern processor's frequency

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
Address sizes:         48 bits physical, 48 bits virtual
CPU(s):                12
On-line CPU(s) list:   0-11
Thread(s) per core:    2
Core(s) per socket:    6
Socket(s):             1
NUMA node(s):          1
Vendor ID:             AuthenticAMD
CPU family:            25
Model:                 80
Model name:            AMD Ryzen 5 5500
Stepping:              0
Frequency boost:       enabled
CPU MHz:               3600.000
CPU max MHz:           3600.0000
CPU min MHz:           1400.0000

```

Take-aways: Challenges and SOTA solutions in the dark silicon era

- Even if we can address all programming challenges, multi-core performance has stopped to scale due to the Dark Silicon problem
- Aggressive dynamic frequency/voltage scaling on CMP to accommodate the demand of **latency**-sensitive, parallelism-limited applications, but the area-efficiency of the slower cores is not great

More cores per chip, slower per core

Export comparison	Intel® Xeon® Platinum 849...	Intel® Xeon® Platinum 846...	Intel® Xeon® Gold 6448H ...	Intel® Xeon® Platinum B44...	Intel® Xeon® Gold 6434H ...
Total Cores	60	48	32	16	8
Total Threads	120	96	64	32	16
Max Turbo Frequency	3.50 GHz	3.80 GHz	4.10 GHz	4.00 GHz	4.10 GHz
Processor Base Frequency	1.90 GHz	2.10 GHz	2.40 GHz	2.90 GHz	3.70 GHz
Cache	112.5 MB	105 MB	50 MB	45 MB	22.5 MB
Intel® UPI Speed	16 GT/s	16 GT/s	16 GT/s	16 GT/s	16 GT/s
Max # of UPI Links	4	4	3	4	3
TDP	350 W	330 W	250 W	270 W	195 W

Xeon Phi

Essentials

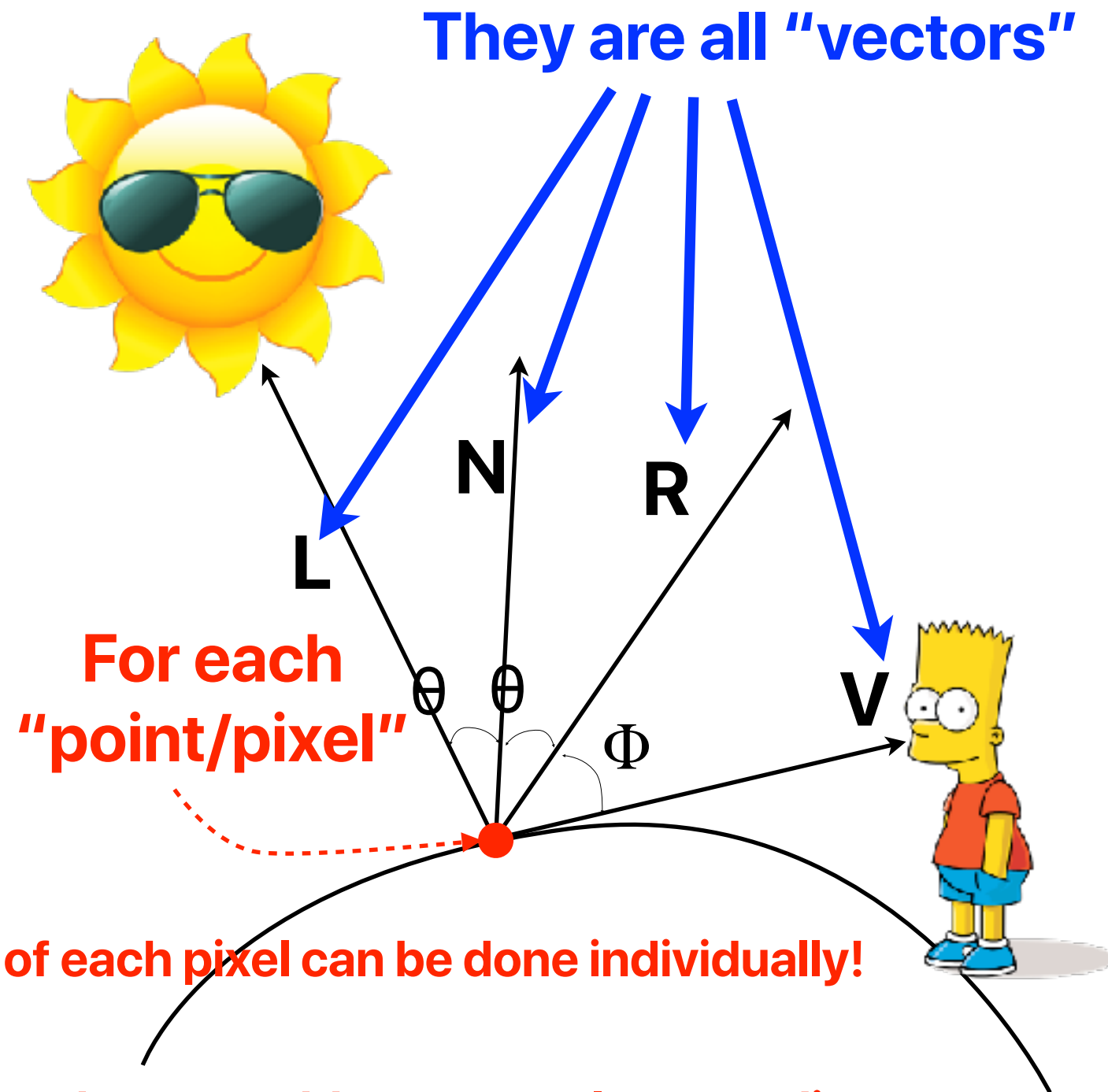
Product Collection	Intel® Xeon Phi™ 72x5 Processor Family
Code Name	Products formerly Knights Mill
Vertical Segment	Server
Processor Number	7295
Off Roadmap	No
Status	Launched
Launch Date ?	Q4'17
Lithography ?	14 nm

Performance

# of Cores ?	72
# of Threads ?	72
Processor Base Frequency ?	1.50 GHz
Max Turbo Frequency ?	1.50 GHz
Cache ?	36 MB L2 Cache
TDP ?	320 W

New type of parallel processors: GPUs

Basic concept of shading



We only need vector add, vector mul, vector div.

$$I_{\text{amb}} = K_{\text{amb}} \cdot M_{\text{amb}}$$

$$I_{\text{diff}} = K_{\text{diff}} \cdot M_{\text{diff}} \cdot (N \cdot L)$$

$$I_{\text{spec}} = K_{\text{spec}} \cdot M_{\text{spec}} \cdot (R \cdot V)^n$$

$$I_{\text{total}} = I_{\text{amb}} + I_{\text{diff}} + I_{\text{spec}}$$

```
void main(void)
{
```

```
    // normalize vectors after interpolation
    vec3 L = normalize(o_toLight);
    vec3 V = normalize(o_toCamera);
    vec3 N = normalize(o_normal);
```

```
    // get Blinn-Phong reflectance components
    float Iamb = ambientLighting();
    float Idif = diffuseLighting(N, L);
    float Ispe = specularLighting(N, L, V);
```

```
    // diffuse color of the object from texture
    vec3 diffuseColor = texture(u_diffuseTexture, o_texcoords)
```

```
    // combination of all components and diffuse color of the
    resultingColor.xyz = diffuseColor * (Iamb + Idif + Ispe);
    resultingColor.a = 1;
```

```
}
```



What GPU architectures should look like?

- Based on the original target of GPU design, what do you think would be reasonable design decisions that GPU architectures make?
 - ① The architecture should render pixels as fast as possible
 - ② The architecture does not need a branch unit
 - ③ The architecture should contain an array of powerful ALUs and functional units that can perform a rich set of operations
 - ④ The architecture should offer very large bandwidth of memory accesses

A. 0

B. 1

C. 2

D. 3

E. 4

```
void main(void) {  
    // normalize vectors after interpolation  
    vec3 L = normalize(o_toLight);  
    vec3 V = normalize(o_toCamera);  
    vec3 N = normalize(o_normal);  
  
    // get Blinn-Phong reflectance components  
    float Iamb = ambientLighting();  
    float Idif = diffuseLighting(N, L);  
    float Ispe = specularLighting(N, L, V);  
  
    // diffuse color of the object from texture  
    vec3 diffuseColor = texture(u_diffuseTexture, o_texcoords).rgb;  
  
    // combination of all components and diffuse color of the object  
    resultingColor.xyz = diffuseColor * (Iamb + Idif + Ispe);  
    resultingColor.a = 1; }  
61
```



What GPU architectures should look like?

- Based on the original target of GPU design, what do you think would be reasonable design decisions that GPU architectures make?
 - ① The architecture should render pixels as fast as possible
 - ② The architecture does not need a branch unit
 - ③ The architecture should contain an array of powerful ALUs and functional units that can perform a rich set of operations
 - ④ The architecture should offer very large bandwidth of memory accesses

A. 0

B. 1

C. 2

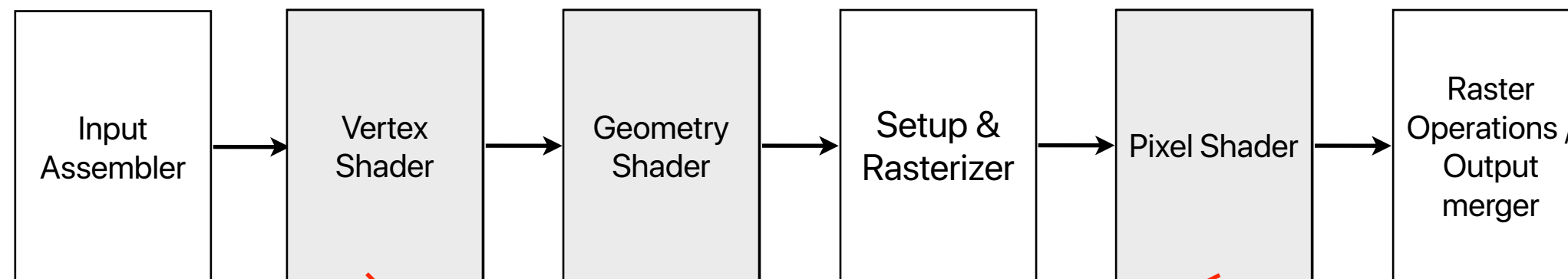
D. 3

E. 4

```
void main(void) {  
    // normalize vectors after interpolation  
    vec3 L = normalize(o_toLight);  
    vec3 V = normalize(o_toCamera);  
    vec3 N = normalize(o_normal);  
  
    // get Blinn-Phong reflectance components  
    float Iamb = ambientLighting();  
    float Idif = diffuseLighting(N, L);  
    float Ispe = specularLighting(N, L, V);  
  
    // diffuse color of the object from texture  
    vec3 diffuseColor = texture(u_diffuseTexture, o_texcoords).rgb;  
  
    // combination of all components and diffuse color of the object  
    resultingColor.xyz = diffuseColor * (Iamb + Idif + Ispe);  
    resultingColor.a = 1; }  
65
```


GPU (Graphics Processing Unit)

- Originally for displaying images $3840 * 2160 * 60 * 3 = 1492992000$ Bytes = 1.39 GB/sec
- UHD video: 3840 * 2160 pixels * 60 frames per second
 - Therefore, GPU is not latency-oriented by design!
 - Even for 120 frames, you still have 8ms latency to get everything done!
- Graphics processing pipeline **1 GHz can give you 8000000 cycles!!!**



These shaders need to be "programmable" to apply different rendering effects/algorithms (Phong shading, Gouraud shading, and etc...)

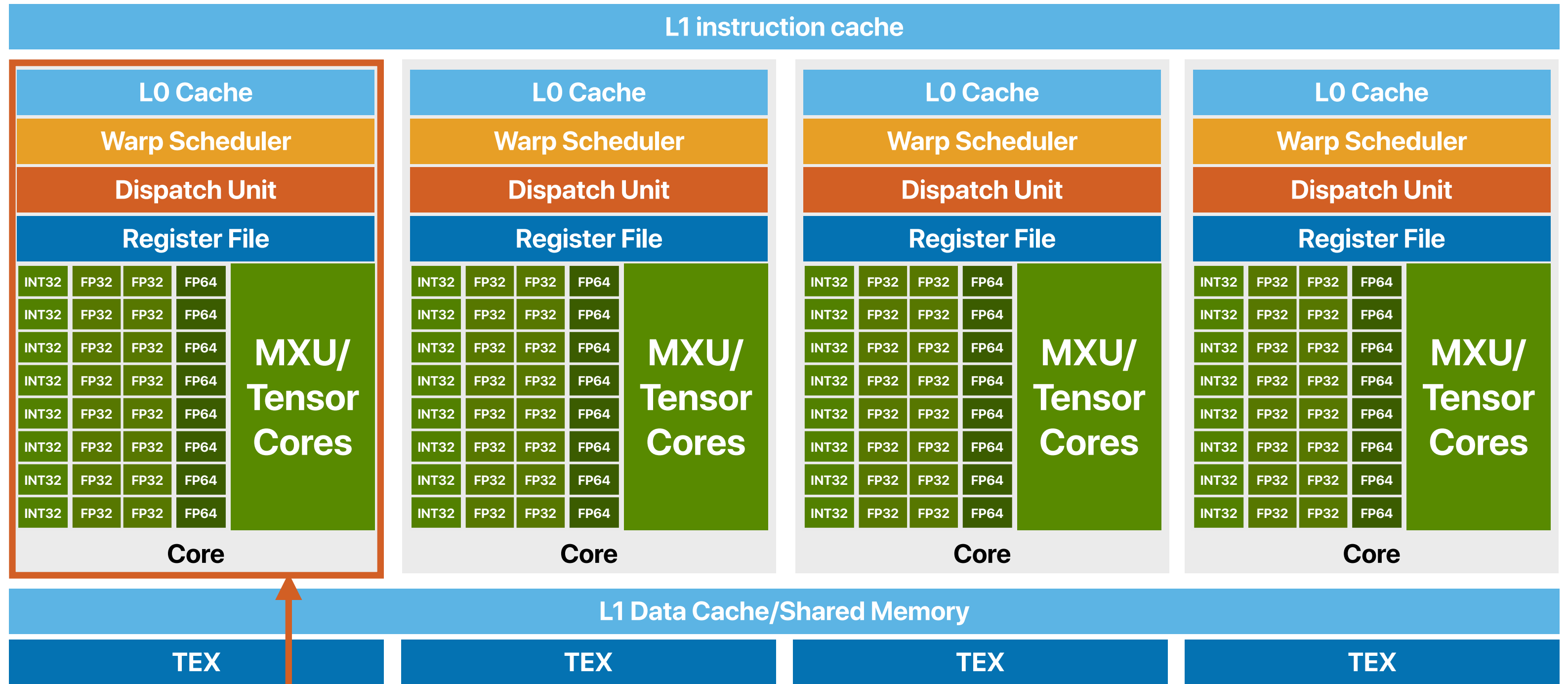
What GPU architectures should look like?

- Based on the original target of GPU design, what do you think would be reasonable design decisions that GPU architectures make?
 - ① The architecture should render pixels as fast as possible
 - ② ✓ The architecture does not need a branch unit
 - ③ The architecture should contain an array of powerful ALUs and functional units that can perform a rich set of operations
 - ✓ ④ The architecture should offer very large bandwidth of memory accesses
- A. 0
B. 1
C. 2
D. 3
E. 4

What's the "appropriate" GPU architecture

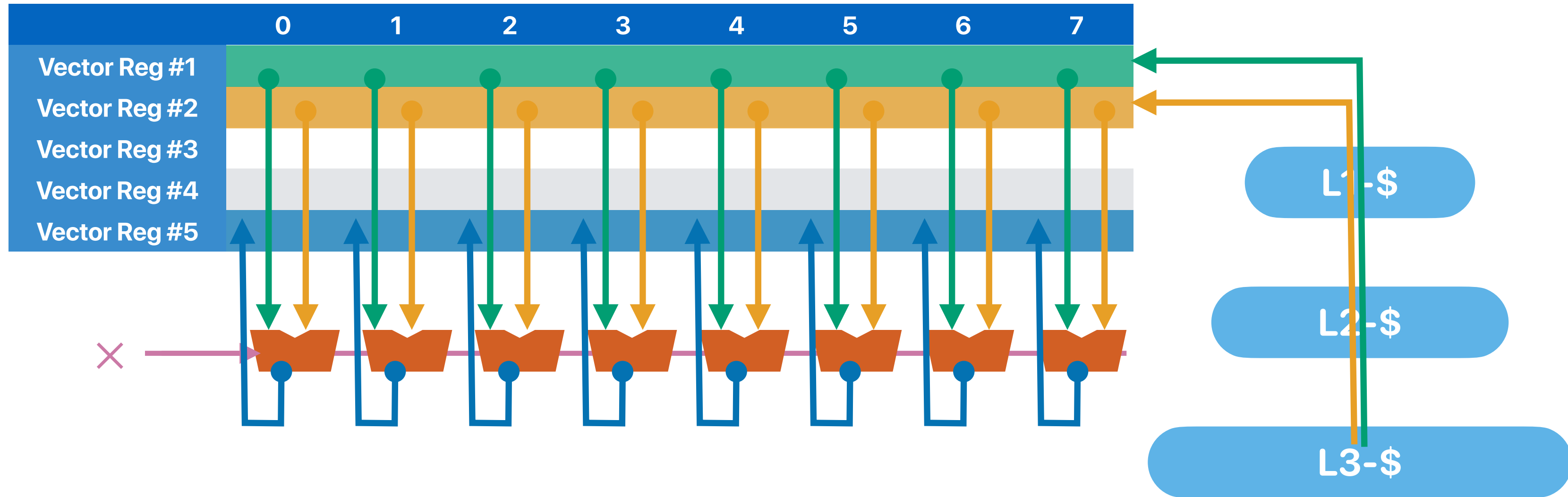
- Lots of ALUs to process pixels in parallel — 2M pixels in HD resolution, very regular workloads
 - Vector processing model
 - Simple operations
 - The ALUs only supports very few instructions
 - Almost no branches
 - Deadline driven and throughput-oriented rather than latency oriented
 - High-bandwidth but also "higher-latency" memory
 - ALUs can be slower
- GPU also follows the idea of slower, but more!**
- 

GPU Architecture



Each core is a "vector processing" unit

Vector processing architecture



Given the same power budget, maximize the efficiency per chip

Slowdown all of them

Chip

0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5

**Low frequency
~0.5W**

=50W!

Chip

Processor Core	Processor Core	Processor Core	Processor Core	Processor Core
Processor Core	Processor Core	Processor Core	Processor Core	Processor Core
Processor Core	Processor Core	Processor Core	Processor Core	Processor Core
Processor Core	Processor Core	Processor Core	Processor Core	Processor Core
Processor Core	Processor Core	Processor Core	Processor Core	Processor Core
Processor Core	Processor Core	Processor Core	Processor Core	Processor Core

**All of them are
smaller, simpler,
lower-frequency,
lower-power cores**

NVIDIA CUDA GPU function

```
__global__ void gpu_matrix_mult(D_TYPE *a, D_TYPE *b,
D_TYPE *c, int size)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    D_TYPE sum = 0;
    for(int i = 0; i < size; i++) {
        sum += a[row * size + i] * b[i * size + col];
    }
    c[row * size + col] = sum;
}
```

threadId

blockId



Parallelism in modern computers

- Instruction-level parallelism — perform various, independent instructions simultaneously
 - Pipeline
 - OoO/Superscalar
- Data-level parallelism — perform the same operation on multiple data elements in parallel
 - SIMD instructions
 - Compute within an SM in GPUs
- Thread-level parallelism — perform independent computation streams (composed of many instructions or SIMD instructions)
 - Multicore/SMT processors
 - Compute using multiple SMs in GPUs

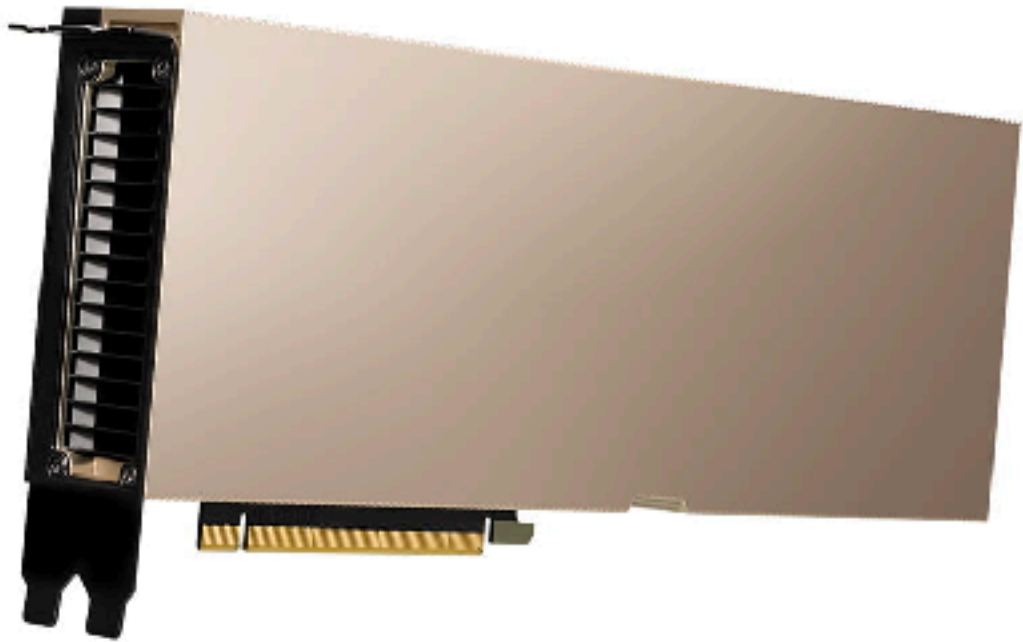
GPUs still suffer from broken Dennard Scaling

NVIDIA Accelerator Specification Comparison

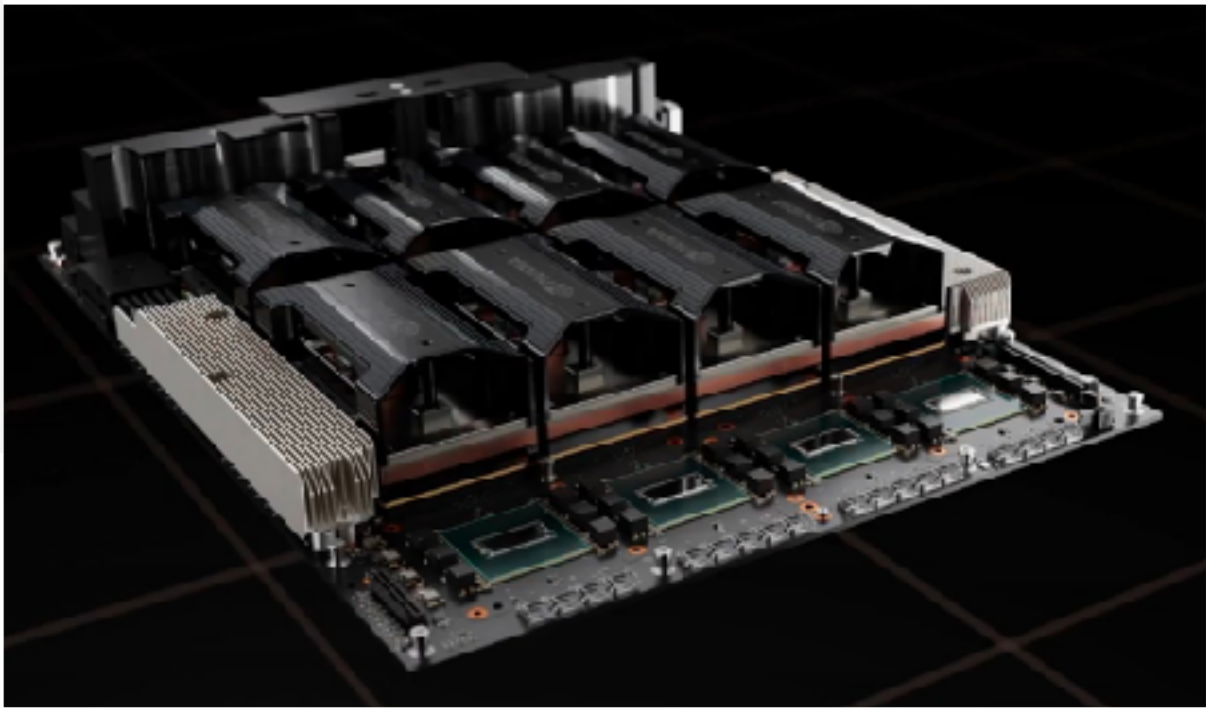
	H100	A100 (80GB)	V100
FP32 CUDA Cores	16896	6912	5120
Tensor Cores	528	432	640
GPU	GH100 (814mm ²)	GA100 (826mm ²)	GV100 (815mm ²)
Transistor Count	80B	54.2B	21.1B
TDP	700W	400W	300W/350W
Manufacturing Process	TSMC 4N	TSMC 7N	TSMC 12nm FFN
Interface	SXM5	SXM4	SXM2/SXM3
Architecture	Hopper	Ampere	Volta

8.75 W/
B Transistors

7.38 W/
B Transistors



<https://www.workstationspecialist.com/product/nvidia-tesla-a100/>



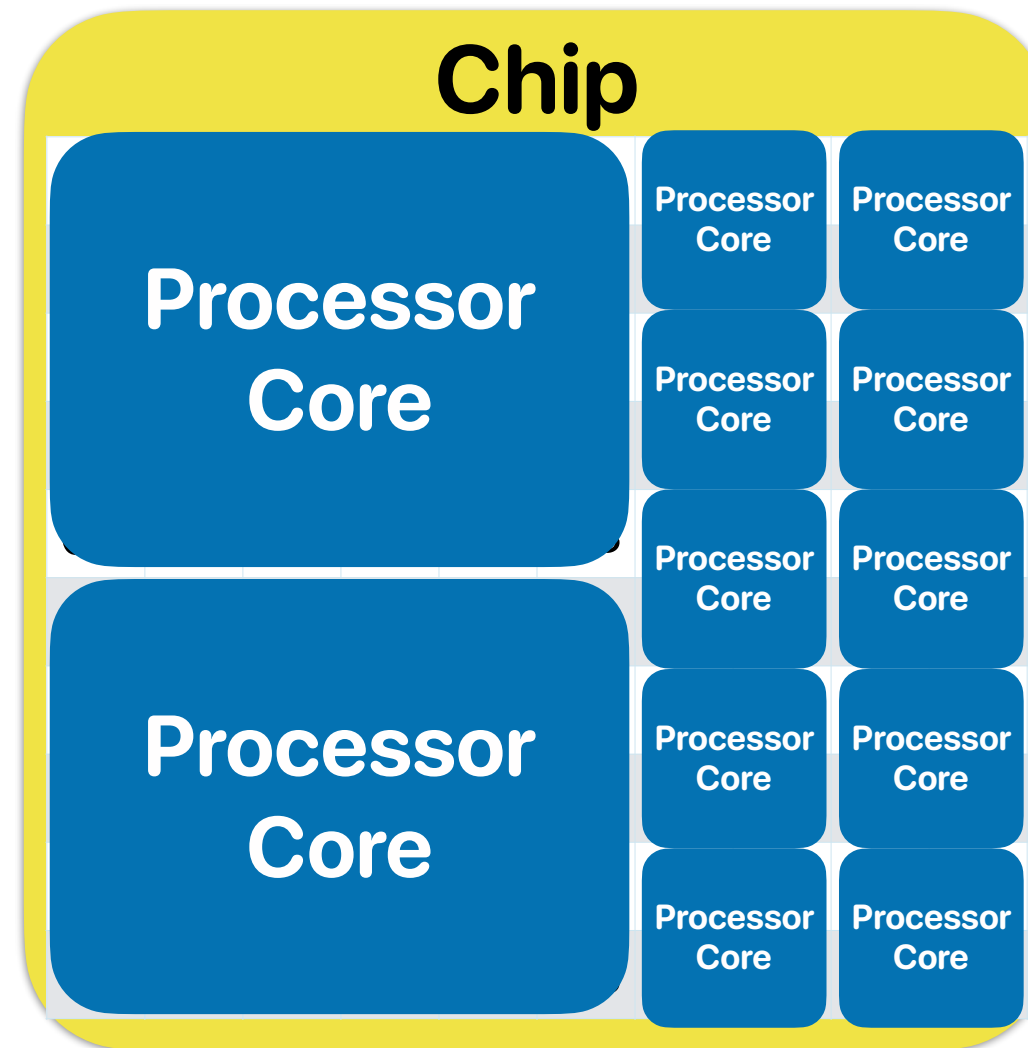
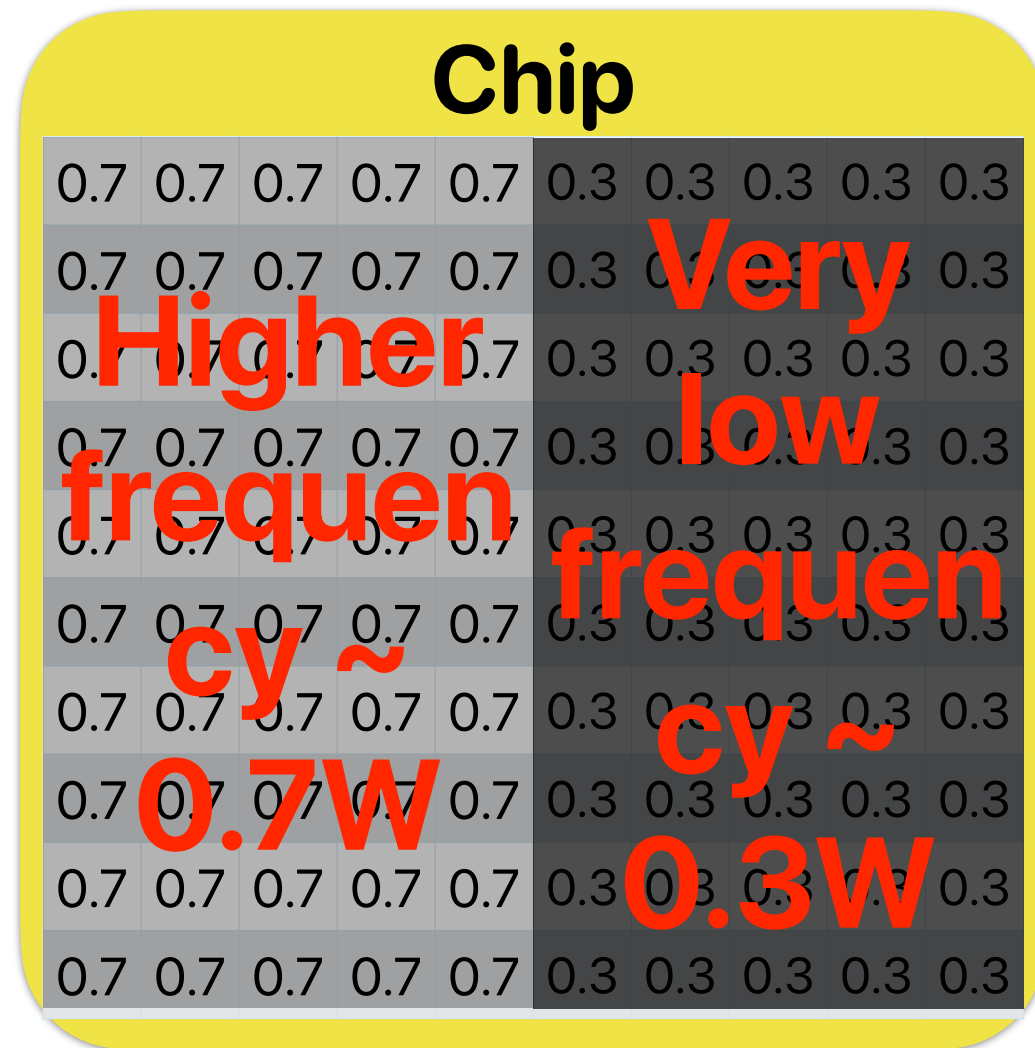
<https://www.servethehome.com/wp-content/uploads/2022/03/NVIDIA-GTC-2022-H100-in-HGX-H100.jpg>

Take-aways: Challenges and SOTA solutions in the dark silicon era

- Even if we can address all programming challenges, multi-core performance has stopped to scale due to the Dark Silicon problem
- Aggressive dynamic frequency/voltage scaling on CMP to accommodate the demand of **latency**-sensitive, parallelism-limited applications, but the area-efficiency of the slower cores is not great
- GPUs/many-core processors improve the **throughput** per-chip through providing massive parallelism where each processing element operates at a lower speed, but not-ideal for latency-sensitive workloads

Given the same power budget, maximize the efficiency per chip

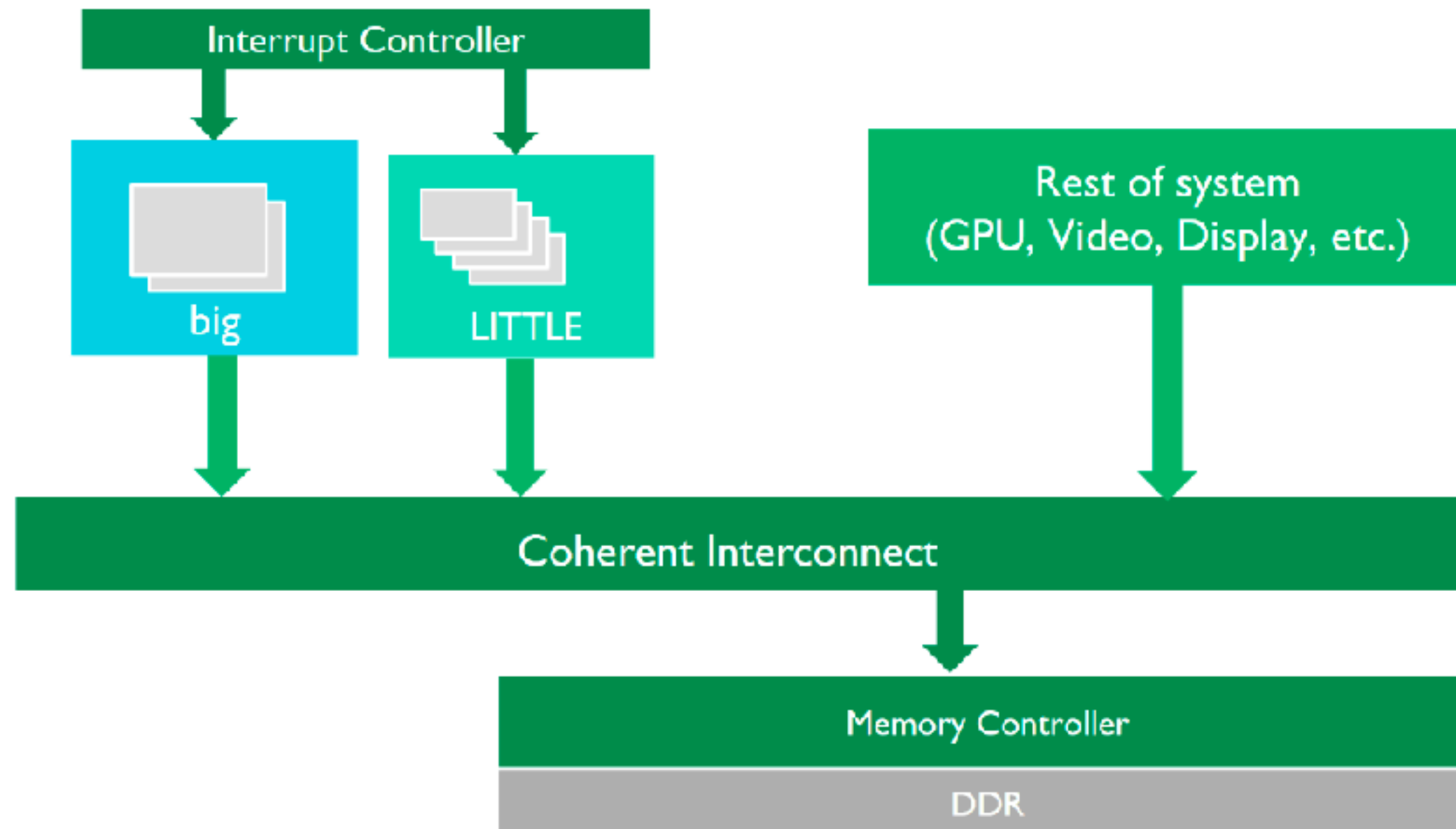
Some faster,
some slower



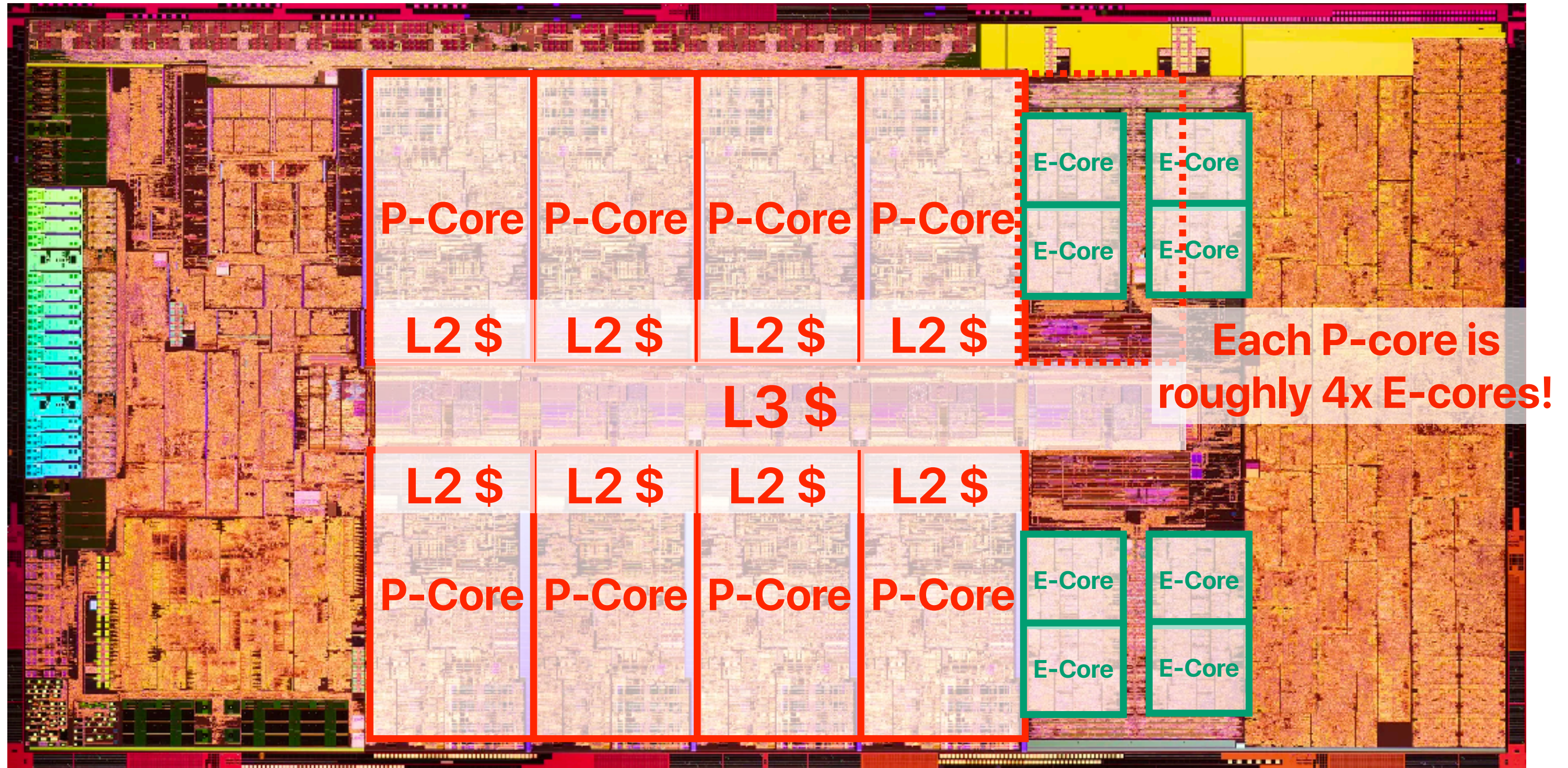
Some powerful cores for
latency sensitive
workloads, many small
cores for highly
parallelizable workloads

Single ISA heterogeneous CMP in ARM's big.LITTLE architecture

big.LITTLE system



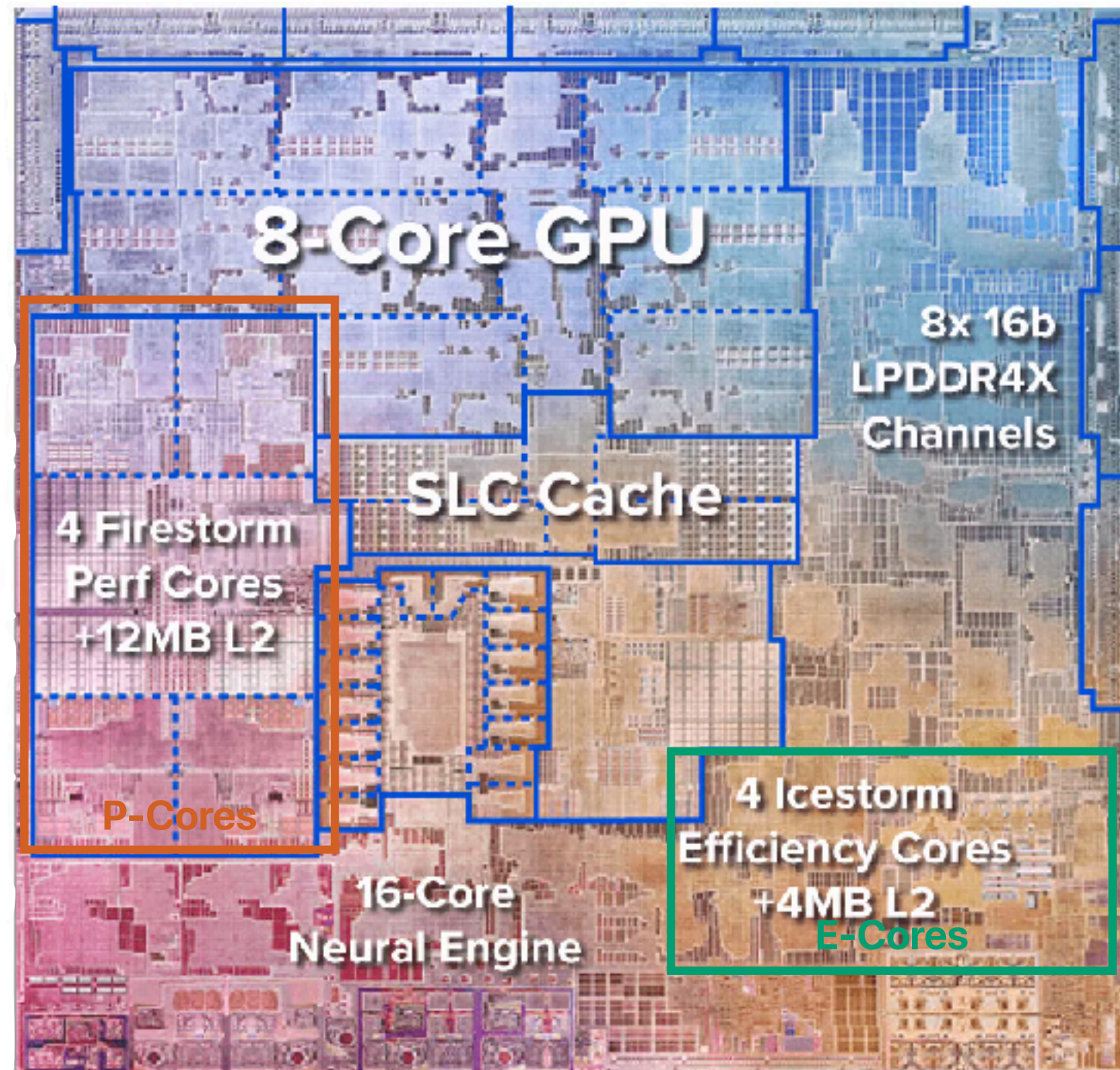
Intel Alder Lake



Single ISA heterogeneous CMP in Intel Processors



Single ISA heterogeneous CMP in Apple's M1





Single ISA heterogeneous CMP (big.Little)

• Assume we have two processor core architectures and the potential configurations within the same chip area:

	Big Only	Little Only	big.Little
P-Cores: higher performance, but higher power consumption and larger size	4		2
E-Cores: lower performance, but lower power consumption and smaller size		24	12

Please identify the correct expectations of their performance

- ① For a program with limited thread-level parallelism, **big.Little** would deliver better than or at least the same level of performance as **Little Only**
- ② For a program with limited thread-level parallelism, **big.Little** would deliver better than or at least the same level of performance as **Big Only**
- ③ For a program with rich thread-level parallelism, **big.Little** would deliver better or at least the same level of performance than **Little Only**
- ④ For a program with rich thread-level parallelism, s **big.Little** would deliver better or at least the same level of performance than **big Only**

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4



Single ISA heterogeneous CMP (big.Little)

- Assume we have two processor core architectures and the potential configurations within the same chip area:

	Big Only	Little Only	big.Little
P-Cores: higher performance, but higher power consumption and larger size	4		2
E-Cores: lower performance, but lower power consumption and smaller size		24	12

Please identify the correct expectations of their performance

- ☒ ① For a program with limited thread-level parallelism, **big.Little** would deliver better than or at least the same level of performance as **Little Only**
- ② For a program with limited thread-level parallelism, **big.Little** would deliver better than or at least the same level of performance as **Big Only**
- ③ For a program with rich thread-level parallelism, **big.Little** would deliver better or at least the same level of performance than **Little Only**
- ☒ ④ For a program with rich thread-level parallelism, **big.Little** would deliver better or at least the same level of performance than **big Only**

A. 0

B. 1

C. 2

D. 3

E. 4

Take-aways: Challenges and SOTA solutions in the dark silicon era

- **Even if we can address all programming challenges, multi-core performance has stopped to scale due to the Dark Silicon problem**
- Aggressive dynamic frequency/voltage scaling on CMP to accommodate the demand of **latency**-sensitive, parallelism-limited applications, but the area-efficiency of the slower cores is not great
- GPUs/many-core processors improve the **throughput** per-chip through providing massive parallelism where each processing element operates at a lower speed, but not-ideal for latency-sensitive workloads
- Single ISA, heterogeneous CMPs (e.g., big.Little cores, Intel/Apple's P-cores/E-cores) find a balance the trade-offs of general-purpose workloads, but won't be ideal if your applications go to either extreme of throughput (massive parallelism) or latency

Announcements

- Last office hour by Hung-Wei — tomorrow
- iEVAL started and ends on 12/05/2025 8am
 - Submit the prove of your participation in course evaluation through Gradescope
 - It can become a full credit reading quiz (it helps to amortize the penalty of another least performing one) — we will drop a total of three now
- **Assignment 5 and programming assignment 3 due this Thursday**
- **Final exam**
 - 170 minutes
 - Two selected questions are considered as CSMS CEP questions — you need to score 60% of those questions to pass the CEP exam

Computer Science & Engineering

203

つづく

